Pseudo Code

```
ALGORITHM: QUICK SORT

QUICKSORT(A, low, high)

1. If low >= high
Return

2. Set key = A[low] // Pivot element

3. Set i = low + 1
Set j = high

4. While i <= j do

While i <= high AND A[i] <= key
i = i + 1

While A[j] > key
j = j - 1

If i < j
Swap(A[i], A[j])

5. End While

6. Swap(A[low], A[j]) // Place pivot at correct position

7. QUICKSORT(A, low, j - 1)

8. QUICKSORT(A, j + 1, high)

9. Return
```

ANALYSIS AND DESIGN OF ALGORITHMS

LABORATORY RECORD

Programs Covered

1. Quick Sort
2. Merge Sort
3. Heap Sort
4. Fractional Knapsack (Greedy Method)
5. 0/1 Knapsack using Dynamic Programming
6. Dijkstra's Algorithm (Single Source Shortest Path)
7. Prim's Algorithm (Minimum Cost Spanning Tree)
8. N-Queens Problem (Backtracking)
9. Floyd-Warshall Algorithm (All Pairs Shortest Path)

10. Strassen's Matrix Multiplication
11. Kruskal's Algorithm (Minimum Cost Spanning Tree)
12. Ford-Fulkerson Algorithm (Maximum Flow)
13. Sum of Subsets Problem (Backtracking)
14. Hamiltonian Cycle (Backtracking)
15. Job Sequencing with Deadlines (Greedy Method)
16. Topological Sorting (using DFS / Stack)
17. Graph Traversal using BFS and DFS
18. Topological Sorting using BFS (Kahn's Algorithm)
19. Travelling Salesman Problem (Dynamic Programming)

1. Quick Sort

Aim

To sort a given set of elements using the Quick Sort method and determine the time taken to sort the elements for different values of n. The elements can be read from a file or can be generated using the random number generator.

Algorithm

1. Start.
2. Read the number of elements n and the array of elements a[].
3. Call QuickSort(a, low, high) with low = 0 and high = n-1.
4. If low >= high, return (base case).
5. Choose key = a[low] as the pivot element.
6. Initialize i = low + 1 and j = high.
7. While i <= j: increment i while a[i] <= key, decrement j while a[j] > key.
8. If i < j, swap a[i] and a[j].
9. When the while loop ends, swap a[key position] with a[j] to place pivot in its correct position.
10. Recursively call QuickSort on the left sub-array (low, j-1) and right sub-array (j+1, high).
11. Repeat until the entire array is sorted.
12. Stop.

C Program

Pseudo Code

```
ALGORITHM: QUICK SORT

QUICKSORT(A, low, high)

1. If low >= high
Return

2. Set key = A[low] // Pivot element

3. Set i = low + 1
Set j = high

4. While i <= j do

While i <= high AND A[i] <= key
i = i + 1

While A[j] > key
j = j - 1

If i < j
Swap(A[i], A[j])

5. End While

6. Swap(A[low], A[j]) // Place pivot at correct position

7. QUICKSORT(A, low, j - 1)

8. QUICKSORT(A, j + 1, high)

9. Return
```

```
#include <stdio.h>
#include <time.h>
```

```
void swap(int *p, int *q)
{
   int temp;
   temp = *p;
   *p = *q;
   *q = temp;
}

void quickSort(int a[], int low, int high)
{
   int i, j, key;
   if (low >= high)
      return;
   key = low;
   i = low + 1;
   j = high;
   while (i <= j)
   {
      while (i <= high && a[i] <= a[key])
         i++;
      while (a[j] > a[key])
         j--;
      if (i < j)
         swap(&a[i], &a[j]);
   }
   swap(&a[key], &a[j]);
   quickSort(a, low, j - 1);
   quickSort(a, j + 1, high);
}

int main()
{
   int n, i, a[1000];
   clock_t start, end;
   double timeTaken;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("Enter elements: \n");
   for (i = 0; i < n; i++)
      scanf("%d", &a[i]);

   start = clock();
   quickSort(a, 0, n - 1);
   end = clock();

   printf("Sorted array: \n");
   for (i = 0; i < n; i++)
      printf("%d ", a[i]);

   timeTaken = (double)(end - start) / CLOCKS_PER_SEC;
   printf("\nTime taken to sort the array is %f seconds\n", timeTaken);

   return 0;
}
```

Sample Input

Enter number of elements: 5
Enter elements:
91 1 49 70 88

Sample Output

Sorted array:
1 49 70 88 91
Time taken to sort the array is 0.000002 seconds

Time Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: $O(n^2)$ (occurs when the array is already sorted or reverse sorted, causing unbalanced partitions)

2. Merge Sort

Aim

To sort a given set of elements using the Merge Sort method and determine the time taken to sort the elements for different values of n. The elements can be read from a file or can be generated using the random number generator.

Algorithm

1. Start.
2. Read the number of elements n and the array of elements a[].
3. Call MergeSort(a, low, high) with low = 0 and high = n-1.
4. If low < high, find mid = (low + high) / 2.
5. Recursively call MergeSort(a, low, mid) to sort the left half.
6. Recursively call MergeSort(a, mid+1, high) to sort the right half.
7. Call Merge(a, low, mid, high) to merge the two sorted halves into a single sorted sequence.
8. In Merge: copy elements into a temporary array by comparing elements of both halves and placing the smaller element first.
9. Copy any remaining elements from either half into the temporary array.
10. Copy the merged elements from the temporary array back into the original array.
11. Stop.

C Program

Pseudo Code

```
ALGORITHM: MERGE SORT

MERGE(A, low, mid, high)

1. Set i = low
Set j = mid + 1
Set k = 0

2. While i <= mid AND j <= high

If A[i] <= A[j]
temp[k] = A[i]
i = i + 1
Else
temp[k] = A[j]
j = j + 1

k = k + 1

3. While i <= mid

temp[k] = A[i]
i = i + 1
k = k + 1

4. While j <= high
```

```
temp[k] = A[j]
j = j + 1
k = k + 1

5. Copy temp[0...k−1] into A[low...high]


MERGESORT(A, low, high)

1. If low < high

mid = (low + high) / 2

2. MERGESORT(A, low, mid)

3. MERGESORT(A, mid + 1, high)

4. MERGE(A, low, mid, high)

5. Return
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int low, int mid, int high)
{
   int i = low, j = mid + 1, k = 0;
   int size = high - low + 1;
   int *temp = (int *)malloc(size * sizeof(int));

   while (i <= mid && j <= high)
   {
      if (a[i] <= a[j])
         temp[k++] = a[i++];
      else
         temp[k++] = a[j++];
   }
   while (i <= mid)
      temp[k++] = a[i++];
   while (j <= high)
      temp[k++] = a[j++];

   for (i = low, k = 0; i <= high; i++, k++)
      a[i] = temp[k];

   free(temp);
}

void mergeSort(int a[], int low, int high)
{
   if (low < high)
   {
      int mid = (low + high) / 2;
      mergeSort(a, low, mid);
      mergeSort(a, mid + 1, high);
      merge(a, low, mid, high);
   }
}
```

```
int main()
{
   int n, i, a[1000];
   clock_t start, end;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("Enter elements: \n");
   for (i = 0; i < n; i++)
      scanf("%d", &a[i]);

   printf("\nUnsorted array: \n");
   for (i = 0; i < n; i++)
      printf("%d ", a[i]);

   start = clock();
   mergeSort(a, 0, n - 1);
   end = clock();

   printf("\n\nSorted array: \n");
   for (i = 0; i < n; i++)
      printf("%d ", a[i]);

   double timeTaken = (double)(end - start) / CLOCKS_PER_SEC;
   printf("\n\nTime taken to sort the array is %f seconds\n", timeTaken);

   printf("\nTime complexity of Merge Sort:\n");
   printf("Best case: O(n log n)\n");
   printf("Average case: O(n log n)\n");
   printf("Worst case: O(n log n)\n");

   return 0;
}
```

Sample Input

Enter number of elements: 5
Enter elements:
91 1 49 70 88

Sample Output

Unsorted array:
91 1 49 70 88

Sorted array:
1 49 70 88 91

Time taken to sort the array is 0.000000 seconds

Time Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

3. Heap Sort

Aim

To sort a given set of elements using the Heap Sort method and determine the time taken to sort the elements for different values of n. The elements can be read from a file or can be generated using the random number generator.

Algorithm

1. Start.
2. Read the number of elements n and the array of elements a[].
3. Build a max heap from the input array by calling Heapify on each non-leaf node, starting from the last non-leaf node (n/2 - 1) down to the root (index 0).
4. In Heapify(a, n, i): find the largest among the node i, its left child (2i+1), and its right child (2i+2).
5. If the largest is not the node i itself, swap a[i] with a[largest] and recursively heapify the affected sub-tree.
6. After the max heap is built, repeatedly swap the root element (maximum) with the last element of the heap.
7. Reduce the heap size by one and call Heapify on the root to restore the max-heap property.
8. Repeat the extraction process until the heap size becomes 1.
9. The array is now sorted in ascending order.
10. Stop.

C Program

```
#include <stdio.h>
#include <time.h>

void heapify(int arr[], int n, int i)
{
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < n && arr[left] > arr[largest])
      largest = left;

   if (right < n && arr[right] > arr[largest])
      largest = right;

   if (largest != i)
   {
      int temp = arr[i];
      arr[i] = arr[largest];
      arr[largest] = temp;

      heapify(arr, n, largest);
   }
}
```

```
void heapSort(int arr[], int n)
{
   for (int i = n / 2 - 1; i >= 0; i--)
      heapify(arr, n, i);

   for (int i = n - 1; i > 0; i--)
   {
      int temp = arr[0];
      arr[0] = arr[i];
      arr[i] = temp;

      heapify(arr, i, 0);
   }
}

int main()
{
   int n, i;
   clock_t start, end;
   double cpu_time;

   printf("Enter number of elements: \n");
   scanf("%d", &n);
   int arr[n];

   srand(time(NULL));
   for (i = 0; i < n; i++)
      arr[i] = rand() % 1000;

   printf("\nUnsorted Array: \n");
   for (i = 0; i < n; i++)
      printf("%d ", arr[i]);

   start = clock();
   heapSort(arr, n);
   end = clock();

   printf("\n\nSorted Numbers: \n");
   for (i = 0; i < n; i++)
      printf("%d ", arr[i]);

   cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
   printf("\n\nExecution Time: %f second\n", cpu_time);

   return 0;
}
```

Pseudo Code

ALGORITHM: HEAP SORT

HEAPIFY(A, n, i)

1. Set largest = i

2. Set left = 2 × i + 1

3. Set right = 2 × i + 2

4. If left < n AND A[left] > A[largest]
largest = left

5. If right < n AND A[right] > A[largest]
largest = right

6. If largest ≠ i

Swap(A[i], A[largest])

HEAPIFY(A, n, largest)

7. Return

Sample Input

Enter number of elements: 5

Sample Output

Unsorted Array:
337 46 642 420 65

Sorted Numbers:
46 65 337 420 642

Execution Time: 0.000007 second

Time Complexity

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

4. Fractional Knapsack (Greedy Method)

Aim

To implement the Fractional Knapsack problem using the Greedy method and find the maximum total profit obtainable.

Algorithm

1. Start.
2. Read the number of items n, the weight and value (profit) of each item, and the knapsack capacity.
3. Compute the ratio (value/weight) for each item.
4. Sort all the items in descending order of their value/weight ratio.
5. Initialize totalValue = 0.
6. For each item in the sorted order: if the item's weight is less than or equal to the remaining capacity, add the entire item (capacity -= weight, totalValue += value).
7. Otherwise, add only the fraction of the item that fits: totalValue += value * (remaining capacity / weight), and set remaining capacity = 0.
8. Repeat until the knapsack is full or all items are considered.
9. Display the total (maximum) value obtained.
10. Stop.

C Program

Pseudo Code

```
ALGORITHM: FRACTIONAL KNAPSACK

FRACTIONAL_KNAPSACK(weights[], values[], n, capacity)

1. For i = 0 to n − 1

ratio[i] = values[i] / weights[i]

2. Sort all items in descending order of ratio

3. Set totalValue = 0

4. For each item in sorted order

If weights[i] <= capacity

totalValue = totalValue + values[i]

capacity = capacity − weights[i]

Else

totalValue = totalValue +
values[i] × (capacity / weights[i])
```

capacity = 0

Break

5. Return totalValue

```
#include <stdio.h>

struct Item
{
   int weight;
   int value;
   float ratio;
};

void sortItems(struct Item items[], int n)
{
   for (int i = 0; i < n - 1; i++)
      for (int j = i + 1; j < n; j++)
         if (items[j].ratio > items[i].ratio)
         {
            struct Item temp = items[i];
            items[i] = items[j];
            items[j] = temp;
         }
}

float knapsack(int weights[], int values[], int n, int capacity)
{
   struct Item items[n];
   for (int i = 0; i < n; i++)
   {
      items[i].weight = weights[i];
      items[i].value = values[i];
      items[i].ratio = (float)values[i] / weights[i];
   }

   sortItems(items, n);

   float totalValue = 0;
   for (int i = 0; i < n; i++)
   {
      if (items[i].weight <= capacity)
      {
         capacity -= items[i].weight;
         totalValue += items[i].value;
      }
      else
      {
         totalValue += items[i].value * ((float)capacity / items[i].weight);
         capacity = 0;
         break;
      }
   }
   return totalValue;
}

int main()
```

```
{
    int n;
    printf("Enter number of items: ");
    scanf("%d", &n);

    int weights[n], values[n];
    printf("Enter weight and value of each item: \n");
    for (int i = 0; i < n; i++)
        scanf("%d %d", &weights[i], &values[i]);

    int capacity;
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

    float maxValue = knapsack(weights, values, n, capacity);
    printf("Maximum value in Knapsack = %.2f\n", maxValue);

    return 0;
}
```

Sample Input

Enter number of items: 3
Enter weight and value of each item:
10 60
20 100
30 120
Enter knapsack capacity: 50

Sample Output

Maximum value in Knapsack = 240.00

Time Complexity

- Time Complexity: O(n log n) (dominated by sorting items by value/weight ratio)
- Space Complexity: O(n)

5. 0/1 Knapsack using Dynamic Programming

Aim

To implement the 0/1 Knapsack problem using Dynamic Programming and find the maximum profit along with the most valuable subset of items.

Algorithm

1. Start.
2. Read the number of items n, weight w[i] and value v[i] for each item, and the knapsack capacity W.
3. Create a DP table T[n+1][W+1] where T[i][j] represents the maximum value obtainable using the first i items with capacity j.
4. Initialize T[0][j] = 0 for all j, and T[i][0] = 0 for all i (base cases).
5. For each item i from 1 to n, and for each capacity j from 0 to W: if w[i-1] <= j, T[i][j] = max(T[i-1][j], v[i-1] + T[i-1][j - w[i-1]]).
6. Otherwise, T[i][j] = T[i-1][j].
7. The maximum profit is given by T[n][W].
8. To find the selected items, trace back from T[n][W]: if T[i][j] != T[i-1][j], item i is included; move to T[i-1][j - w[i-1]], else move to T[i-1][j].
9. Display the maximum value and the list of selected items.
10. Stop.

C Program

Pseudo Code

```
01_KNAPSACK(weights[], values[], n, W)

1. Create dp[0...n][0...W] and initialize all elements to 0.

2. For i = 1 to n

For j = 0 to W

If weights[i − 1] <= j

dp[i][j] = MAX(values[i − 1] +
dp[i − 1][j − weights[i − 1]],
dp[i − 1][j])

Else

dp[i][j] = dp[i − 1][j]

3. Set maxProfit = dp[n][W]

4. Set j = W

5. For i = n down to 1

If dp[i][j] ≠ dp[i − 1][j]

Mark item i as selected

j = j − weights[i − 1]

6. Return maxProfit and selected items
```

MAIN PROGRAM

1. Read n

2. Read weights[]

3. Read values[]

4. Read W (Knapsack Capacity)

5. Call 01_KNAPSACK(weights, values, n, W)

6. Print DP table

7. Print maximum profit

8. Print selected items

```
#include <stdio.h>

int max(int a, int b)
{
   return (a > b) ? a : b;
}

int knapsack(int capacity, int weight[], int value[], int n, int dp[][50])
{
   for (int i = 0; i <= n; i++)
   {
      for (int w = 0; w <= capacity; w++)
      {
         if (i == 0 || w == 0)
            dp[i][w] = 0;
         else if (weight[i - 1] <= w)
            dp[i][w] = max(value[i - 1] + dp[i - 1][w - weight[i - 1]], dp[i - 1][w]);
         else
            dp[i][w] = dp[i - 1][w];
      }
   }
   return dp[n][capacity];
}

int main()
{
   int n, capacity;
   printf("Enter number of items: ");
   scanf("%d", &n);

   int weight[n], value[n];
   printf("Enter weight and value/profit of each item: \n");
   for (int i = 0; i < n; i++)
      scanf("%d %d", &weight[i], &value[i]);

   printf("Enter Knapsack capacity: ");
   scanf("%d", &capacity);

   int dp[n + 1][50];
   int maxValue = knapsack(capacity, weight, value, n, dp);
```

```
    printf("\nThe table is: \n");
    for (int i = 0; i <= n; i++)
    {
        for (int w = 0; w <= capacity; w++)
            printf("%d ", dp[i][w]);
        printf("\n");
    }

    printf("\nThe maximum profit is %d\n", maxValue);
    printf("\nThe most valuable subset is: ");

    int w = capacity;
    for (int i = n; i > 0; i--)
    {
        if (dp[i][w] != dp[i - 1][w])
        {
            printf("item %d ", i);
            w = w - weight[i - 1];
        }
    }
    printf("\n");

    return 0;
}
```

Sample Input

Enter number of items: 2
Enter weight and value/profit of each item:
4 3
3 4
Enter Knapsack capacity: 4

Sample Output

The table is:
0 0 0 0 0
0 0 0 0 3
0 0 0 4 4

The maximum profit is 4

The most valuable subset is: item 2

Time Complexity

- Time Complexity: O(n * W) where n = number of items, W = knapsack capacity
- Space Complexity: O(n * W)

6. Dijkstra's Algorithm (Single Source Shortest Path)

Aim

To find the shortest paths from a given source vertex to all other vertices in a weighted graph using Dijkstra's algorithm.

Algorithm

1. Start.
2. Read the number of nodes n and the cost adjacency matrix cost[][] of the graph (cost[i][j] = INFINITY if there is no edge between i and j).
3. Read the source vertex v.
4. Initialize dist[i] = cost[v][i] for all vertices i, and flag[i] = 0 for all vertices (flag[i] = 1 indicates the shortest path to i has been finalized).
5. Set flag[v] = 1 and count = 1.
6. While count < n: find the vertex u with the minimum dist[u] among all vertices with flag[u] = 0.
7. Set flag[u] = 1 and increment count.
8. For every vertex w with flag[w] = 0: if dist[u] + cost[u][w] < dist[w], update dist[w] = dist[u] + cost[u][w].
9. Repeat until all vertices are finalized.
10. Display the shortest distance from the source to every vertex.
11. Stop.

C Program

Pseudo Code

```
ALGORITHM: DIJKSTRA'S ALGORITHM (Shortest Path)

DIJKSTRA(cost[][], n, source)

1. For i = 1 to n

dist[i] = cost[source][i]

flag[i] = 0

2. Set flag[source] = 1.

3. Set dist[source] = 0.

4. Set count = 2.

5. While count <= n do

a. Find the vertex u having the minimum dist[u]
such that flag[u] = 0.

b. Set flag[u] = 1.

c. count = count + 1.
```

d. For each vertex w such that flag[w] = 0

If dist[u] + cost[u][w] < dist[w] then

dist[w] = dist[u] + cost[u][w]

6. Return the distance array (dist[]).

```
#include <stdio.h>
#define INFINITY 9999

void dijkstra(int n, int v, int cost[10][10], int dist[10])
{
   int i, u, w, count, min, flag[10];

   for (i = 1; i <= n; i++)
   {
      flag[i] = 0;
      dist[i] = cost[v][i];
   }

   flag[v] = 1;
   count = 2;

   while (count <= n)
   {
      min = INFINITY;
      for (w = 1; w <= n; w++)
         if (dist[w] < min && !flag[w])
         {
            min = dist[w];
            u = w;
         }

      flag[u] = 1;
      count++;

      for (w = 1; w <= n; w++)
         if ((dist[u] + cost[u][w] < dist[w]) && !flag[w])
            dist[w] = dist[u] + cost[u][w];
   }
}

int main()
{
   int n, v, i, j, cost[10][10], dist[10];

   printf("\nEnter the Number of Nodes: ");
   scanf("%d", &n);

   printf("Enter the cost matrix: \n");
   for (i = 1; i <= n; i++)
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = INFINITY;
```

```
    }

  printf("\nEnter the Source node: ");
  scanf("%d", &v);

  dijkstra(n, v, cost, dist);

  printf("\nShortest Path: \n");
  for (i = 1; i <= n; i++)
    printf("%d -> %d, cost = %d\n", v, i, dist[i]);

  printf("\nTime Complexity: O((V+E) log V)\n");

  return 0;
}
```

Sample Input

Enter the Number of Nodes: 4
Enter the cost matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Enter the Source node: 1

Sample Output

Shortest Path:
1 -> 1, cost = 0
1 -> 2, cost = 2
1 -> 3, cost = 5
1 -> 4, cost = 6

Time Complexity

- Time Complexity: $O(V^2)$ using adjacency matrix, or O((V+E) log V) using a priority queue/min-heap

7. Prim's Algorithm (Minimum Cost Spanning Tree)

Aim

To find the Minimum Cost Spanning Tree of a given undirected weighted graph using Prim's algorithm.

Algorithm

1. Start.
2. Read the number of nodes n and the cost adjacency matrix cost[][] of the graph (cost[i][j] = INFINITY if there is no edge).
3. Mark vertex 1 as visited; all other vertices are unvisited.
4. Initialize mincost = 0.
5. Repeat until all vertices are visited (n-1 edges are added): search the entire cost matrix for the minimum cost edge (a, b) such that exactly one of a or b is visited and the other is not.
6. Add this edge to the spanning tree, display the edge and its cost, and add the cost to mincost.
7. Mark the newly included vertex as visited.
8. Repeat the search-and-add process until all vertices are included in the spanning tree.
9. Display the total minimum cost of the spanning tree.
10. Stop.

C Program

Pseudo Code

```
PRIM(cost[][], n)

1. Set visited[1] = 1.

2. Set all other visited vertices to 0.

3. Set mincost = 0.

4. Set edgesAdded = 0.

5. While edgesAdded < n - 1 do

a. Set min = INFINITY.

b. For i = 1 to n
For j = 1 to n
If visited[i] = 1 AND visited[j] = 0 AND cost[i][j] < min then
min = cost[i][j]
a = i
b = j

c. Print the edge (a, b) and its cost (min).

d. mincost = mincost + min.

e. Set visited[b] = 1.

f. Set cost[a][b] = INFINITY.

g. Set cost[b][a] = INFINITY.
```

h. edgesAdded = edgesAdded + 1.

6. Print the total minimum cost (mincost).

```
#include <stdio.h>
#define INFINITY 9999
#define MAX 10

int main()
{
   int cost[MAX][MAX], visited[MAX] = {0};
   int n, i, j, a = 0, b = 0, u = 0, v = 0, min, mincost = 0;

   printf("Enter the number of nodes: ");
   scanf("%d", &n);

   printf("Enter the adjacency matrix: \n");
   for (i = 1; i <= n; i++)
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = INFINITY;
      }

   visited[1] = 1;
   printf("\n");

   while (u < n - 1)
   {
      min = INFINITY;
      for (i = 1; i <= n; i++)
      {
         for (j = 1; j <= n; j++)
         {
            if (cost[i][j] < min)
            {
               if (visited[i] != 0 || visited[j] == 0)
               {
                  if (!(visited[i] == 0 && visited[j] == 0))
                  {
                     min = cost[i][j];
                     a = i;
                     b = j;
                  }
               }
            }
         }
      }

      if (visited[a] == 0 || visited[b] == 0)
      {
         printf("Edge %d: (%d, %d) cost = %d\n", ++u, a, b, min);
         mincost += min;
         visited[b] = 1;
      }
```

```
      cost[a][b] = cost[b][a] = INFINITY;
    }

    printf("\nMinimum cost = %d\n", mincost);
    return 0;
}
```

Sample Input

Enter the number of nodes: 4
Enter the adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0

Sample Output

Edge 1: (1, 2) cost = 2
Edge 2: (2, 3) cost = 3
Edge 3: (1, 4) cost = 6

Minimum cost = 11

Time Complexity

- Time Complexity: $O(V^2)$ using adjacency matrix, or $O(E \log V)$ using a priority queue/min-heap

8. N-Queens Problem (Backtracking)

Aim

To implement the N-Queens problem using the Backtracking method and print all possible solutions for placing N queens on an N x N chessboard such that no two queens attack each other.

Algorithm

1. Start.
2. Read the number of queens n.
3. Define array a[] where a[i] stores the column position of the queen placed in row i.
4. Start placing the queen for row k = 1.
5. Function Place(k, i) checks if placing a queen in row k, column i is safe: for each row j from 1 to k-1, check that a[j] != i (no same column) and |a[j] - i| != |j - k| (no same diagonal).
6. If safe, return true; otherwise return false.
7. For the current row k, try columns i = 1 to n. If Place(k, i) is true, set a[k] = i.
8. If k == n, a complete solution is found; print the board configuration and increment the solution count.
9. Otherwise, move to the next row (k = k+1) and repeat the placement process.
10. If no column in the current row is safe, backtrack to the previous row and try the next column there.
11. Repeat until all possible placements (solutions) have been explored.
12. Display the total number of solutions found.
13. Stop.

C Program

Pseudo Code

```
ALGORITHM: N-QUEENS (Backtracking)

PLACE(k, i)

1. For j = 1 to k - 1

If a[j] = i OR |a[j] - i| = |j - k| then

Return FALSE

2. Return TRUE


NQUEENS(n)

1. Set k = 1.

2. Set a[k] = 0.

3. While k ≠ 0 do

a. Set i = a[k] + 1.
```

b. While i ≤ n AND PLACE(k, i) = FALSE do

i = i + 1.

c. If i ≤ n then

a[k] = i.

If k = n then

Print the current solution.

Else

k = k + 1.

a[k] = 0.

d. Else

k = k - 1. // Backtrack

4. Repeat until all solutions are found.

```
#include <stdio.h>
#include <math.h>

int a[30], count = 0;

int place(int k, int i)
{
   int j;
   for (j = 1; j < k; j++)
   {
      if (a[j] == i || abs(a[j] - i) == abs(j - k))
         return 0;
   }
   return 1;
}

void printSolution(int n)
{
   int i, j;
   printf("\nSolution #%d: \n", ++count);
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         if (a[i] == j)
            printf("Q ");
         else
            printf("* ");
      }
      printf("\n");
   }
}

void nQueens(int n)
```

```
{
   int k = 1, i;
   a[k] = 0;

   while (k != 0)
   {
      i = a[k] + 1;
      while (i <= n && !place(k, i))
         i++;

      if (i <= n)
      {
         a[k] = i;
         if (k == n)
            printSolution(n);
         else
         {
            k++;
            a[k] = 0;
         }
      }
      else
         k--;
   }
}

int main()
{
   int n;
   printf("Enter the number of queens: ");
   scanf("%d", &n);

   nQueens(n);

   printf("\nTotal solutions = %d\n", count);
   printf("\nTime Complexity: O(N!)\n");

   return 0;
}
```

Sample Input

Enter the number of queens: 4

Sample Output

Solution #1:
* Q * *
* * * Q
Q * * *
* * Q *

Solution #2:

```
* * Q *
Q * * *
* * * Q
* Q * *
```

Total solutions = 2

Time Complexity

- Time Complexity: O(N!) (worst case, with backtracking pruning unsafe branches early)

9. Floyd-Warshall Algorithm (All Pairs Shortest Path)

Aim

To find the shortest distance between every pair of vertices in a given weighted directed graph using Floyd-Warshall's algorithm.

Algorithm

1. Start.
2. Read the number of vertices n and the cost adjacency matrix a[][] (use a large value INF if there is no direct edge between two vertices, and 0 on the diagonal).
3. Initialize the distance matrix D as D[0] = a (the original cost matrix).
4. For k = 1 to n (k represents the intermediate vertex allowed): for each pair of vertices i and j, update D[i][j] = min(D[i][j], D[i][k] + D[k][j]).
5. This means: if going from i to j through the intermediate vertex k gives a shorter path than the current known path, update the distance.
6. Repeat the above step for every vertex k from 1 to n, progressively allowing more intermediate vertices.
7. After considering all n vertices as intermediates, D contains the shortest distance between every pair of vertices.
8. Display the final shortest path matrix.
9. Stop.

C Program

Pseudo Code

```
ALGORITHM: FLOYD-WARSHALL ALGORITHM (All-Pairs Shortest Path)

FLOYD_WARSHALL(D[][], n)

1. For k = 0 to n - 1

For i = 0 to n - 1

For j = 0 to n - 1

If D[i][k] + D[k][j] < D[i][j] then

D[i][j] = D[i][k] + D[k][j]

2. Return the updated distance matrix D[][].


MAIN PROGRAM

1. Read the number of vertices (n).

2. Read the adjacency matrix (D[][]).

3. For every pair of vertices (i, j)

If there is no edge between i and j and i ≠ j
```

Set D[i][j] = INFINITY.

4. Call:
FLOYD_WARSHALL(D, n)

5. Display the all-pairs shortest path matri

```
#include <stdio.h>
#define INF 9999

int min(int a, int b)
{
   return (a < b) ? a : b;
}

void floyd(int p[10][10], int n)
{
   int i, j, k;
   for (k = 0; k < n; k++)
      for (i = 0; i < n; i++)
         for (j = 0; j < n; j++)
            p[i][j] = min(p[i][j], p[i][k] + p[k][j]);
}

int main()
{
   int a[10][10], n, i, j;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix: \n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
      {
         scanf("%d", &a[i][j]);
         if (a[i][j] == 0 && i != j)
            a[i][j] = INF;
      }

   floyd(a, n);

   printf("\nShortest path matrix: \n");
   for (i = 0; i < n; i++)
   {
      for (j = 0; j < n; j++)
         printf("%d ", a[i][j]);
      printf("\n");
   }

   printf("\nTime Complexity: O(n^3)\n");
   printf("Space Complexity: O(n^2)\n");

   return 0;
}
```

Sample Input

Enter number of vertices: 4
Enter adjacency matrix:
0 3 9999 9999
2 0 9999 1
9999 7 0 9999
6 9999 9999 0

Sample Output

Shortest path matrix:
0 3 9999 4
2 0 9999 1
9 6 0 7
6 9 9999 0

Time Complexity

- Time Complexity: $O(n^3)$
- Space Complexity: $O(n^2)$

10. Strassen's Matrix Multiplication

Aim

To multiply two n x n matrices using Strassen's matrix multiplication algorithm, which reduces the number of multiplications required compared to the conventional method.

Algorithm

1. Start.
2. Read two n x n matrices A and B (n is a power of 2).
3. Divide each matrix A and B into four equal sub-matrices of size n/2 x n/2: A11, A12, A21, A22 and B11, B12, B21, B22.
4. Compute the following 7 matrix products (instead of the usual 8): M1 = (A11+A22)(B11+B22); M2 = (A21+A22)*B11; M3 = A11*(B12-B22); M4 = A22*(B21-B11); M5 = (A11+A12)*B22; M6 = (A21-A11)*(B11+B12); M7 = (A12-A22)*(B21+B22).
5. Compute the four quadrants of the result matrix C using these products: C11 = M1+M4-M5+M7; C12 = M3+M5; C21 = M2+M4; C22 = M1-M2+M3+M6.
6. Combine C11, C12, C21, C22 to form the final result matrix C = A x B.
7. If the sub-matrices are larger than the base case size, recursively apply the same procedure to compute each of the 7 products.
8. Stop.

C Program

Pseudo Code

```
ALGORITHM: STRASSEN'S MATRIX MULTIPLICATION

STRASSEN(A[][], B[][], n)

1. If n = 1 then

Return A × B.

2. Partition matrix A into four sub-matrices:
A11, A12, A21, A22.

3. Partition matrix B into four sub-matrices:
B11, B12, B21, B22.

4. Compute the seven products:

M1 = (A11 + A22) × (B11 + B22)

M2 = (A21 + A22) × B11

M3 = A11 × (B12 − B22)

M4 = A22 × (B21 − B11)

M5 = (A11 + A12) × B22

M6 = (A21 − A11) × (B11 + B12)
```

M7 = (A12 − A22) × (B21 + B22)

5. Compute the result sub-matrices:

C11 = M1 + M4 − M5 + M7

C12 = M3 + M5

C21 = M2 + M4

C22 = M1 − M2 + M3 + M6

6. Combine C11, C12, C21, and C22 to form the final matrix C.

7. Return matrix C.

```
#include <stdio.h>
#define N 2

void addMatrix(int a[N][N], int b[N][N], int result[N][N])
{
   for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
         result[i][j] = a[i][j] + b[i][j];
}

void subMatrix(int a[N][N], int b[N][N], int result[N][N])
{
   for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
         result[i][j] = a[i][j] - b[i][j];
}

void strassenMultiply(int A[N][N], int B[N][N], int C[N][N])
{
   int M1, M2, M3, M4, M5, M6, M7;

   /* For a 2x2 base case, treat each element directly */
   M1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
   M2 = (A[1][0] + A[1][1]) * B[0][0];
   M3 = A[0][0] * (B[0][1] - B[1][1]);
   M4 = A[1][1] * (B[1][0] - B[0][0]);
   M5 = (A[0][0] + A[0][1]) * B[1][1];
   M6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
   M7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);

   C[0][0] = M1 + M4 - M5 + M7;
   C[0][1] = M3 + M5;
   C[1][0] = M2 + M4;
   C[1][1] = M1 - M2 + M3 + M6;
}

int main()
{
   int A[N][N], B[N][N], C[N][N];

   printf("Enter elements of matrix A (%dx%d): \n", N, N);
```

```
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &A[i][j]);

    printf("Enter elements of matrix B (%dx%d): \n", N, N);
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &B[i][j]);

    strassenMultiply(A, B, C);

    printf("\nResultant matrix C = A x B: \n");
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
            printf("%d ", C[i][j]);
        printf("\n");
    }

    printf("\nTime Complexity: O(n^2.81)\n");

    return 0;
}
```

Sample Input

Enter elements of matrix A (2x2):
1 2
3 4
Enter elements of matrix B (2x2):
5 6
7 8

Sample Output

Resultant matrix C = A x B:
19 22
43 50

Time Complexity

- Time Complexity: O(n^2.81) (better than the conventional $O(n^3)$ matrix multiplication)

11. Kruskal's Algorithm (Minimum Cost Spanning Tree)

Aim

To find the Minimum Cost Spanning Tree of a given undirected weighted, connected graph using Kruskal's algorithm.

Algorithm

1. Start.
2. Read the number of vertices and edges, and the weight of each edge (u, v, w).
3. Sort all the edges of the graph in non-decreasing (ascending) order of their weight.
4. Initialize each vertex as a separate set using the Union-Find (Disjoint Set) data structure, where parent[i] = i for all i.
5. Initialize the minimum cost spanning tree cost to 0 and the count of edges included to 0.
6. For each edge (u, v, w) in the sorted order: find the representative (root) of the sets containing u and v using the Find operation.
7. If the roots are different (i.e., u and v belong to different sets / including the edge does not form a cycle), include this edge in the MST, add w to the total cost, and perform a Union of the two sets.
8. If the roots are the same, discard this edge (including it would form a cycle).
9. Repeat the process for all edges until (n-1) edges have been included in the spanning tree, where n is the number of vertices.
10. Display all the selected edges and the total minimum cost.
11. Stop.

C++ Program

Pseudo Code

```
ALGORITHM: KRUSKAL'S ALGORITHM (Minimum Spanning Tree)

FIND(parent[], x)

1. If parent[x] = x then

Return x.

2. Return FIND(parent, parent[x]).


KRUSKAL(edges[], V, E)

1. For i = 0 to V - 1

parent[i] = i.

2. Sort all edges in ascending order of their weights.

3. Set cost = 0.

4. Set edgesUsed = 0.
```

5. For each edge (u, v, w) in the sorted list

a. a = FIND(parent, u)

b. b = FIND(parent, v)

c. If a ≠ b then

Print edge (u, v) with weight w.

cost = cost + w.

parent[a] = b. // Union operation

edgesUsed = edgesUsed + 1.

d. If edgesUsed = V - 1 then

Break.

6. Print the minimum cost.

```
#include <iostream>
using namespace std;

struct Edge
{
   int u, v, w;
};

int parent[10];

int find(int x)
{
   if (parent[x] == x)
      return x;
   return find(parent[x]);
}

int main()
{
   Edge edge[] = {
      {0, 1, 1},
      {1, 2, 3},
      {0, 2, 5},
      {2, 3, 4},
      {1, 3, 3}
   };

   int vertices = 4;
   int e = 5;

   for (int i = 0; i < vertices; i++)
      parent[i] = i;

   /* sort edges in ascending order of weight (bubble sort) */
   for (int i = 0; i < e - 1; i++)
      for (int j = i + 1; j < e; j++)
```

```
            if (edge[j].w < edge[i].w)
            {
               Edge temp = edge[i];
               edge[i] = edge[j];
               edge[j] = temp;
            }

      cout << "Edges in MST: \n";
      int cost = 0;
      int edgesUsed = 0;

      for (int i = 0; i < e && edgesUsed < vertices - 1; i++)
      {
         int a = find(edge[i].u);
         int b = find(edge[i].v);

         if (a != b)
         {
            cout << edge[i].u << " - " << edge[i].v << " - " << edge[i].w << endl;
            cost = cost + edge[i].w;
            parent[a] = b;
            edgesUsed++;
         }
      }

      cout << "Minimum cost: " << cost << endl;
      return 0;
   }
```

Sample Input

Graph with 4 vertices and 5 edges: (0,1,1) (1,2,3) (0,2,5) (2,3,4) (1,3,3)

Sample Output

Edges in MST:
0 - 1 - 1
1 - 2 - 3
1 - 3 - 3
Minimum cost: 7

Time Complexity

- Time Complexity: O(E log E) (dominated by sorting the edges; E = number of edges)

12. Ford-Fulkerson Algorithm (Maximum Flow)

Aim

To find the maximum flow in a given flow network from a source vertex to a sink vertex using the Ford-Fulkerson method.

Algorithm

1. Start.
2. Read the number of vertices and the capacity matrix graph[][] of the flow network, along with the source vertex s and sink vertex t.
3. Initialize max_flow = 0 and create a residual graph rGraph[][] which is initially the same as the given capacity graph.
4. While there exists an augmenting path from s to t in the residual graph (found using BFS): mark all vertices as unvisited, push the source into a queue and mark it visited.
5. Perform BFS: for the current vertex u, examine every vertex v; if v is not visited and the residual capacity rGraph[u][v] > 0, mark v visited, set parent[v] = u, and enqueue v.
6. If the sink t is reached during BFS, an augmenting path exists.
7. Find path_flow as the minimum residual capacity among all the edges along the path from s to t (traced using the parent[] array).
8. Update the residual capacities of the edges and reverse edges along the path: for every edge (u, v) in the path, rGraph[u][v] -= path_flow and rGraph[v][u] += path_flow.
9. Add path_flow to max_flow.
10. Repeat the process until no augmenting path exists from s to t in the residual graph.
11. Display the maximum flow.
12. Stop.

C Program

Pseudo Code

```
ALGORITHM: FORD-FULKERSON ALGORITHM (Maximum Flow)

BFS(rGraph[][], s, t, parent[])

1. Initialize all visited[] = FALSE.

2. Create an empty queue.

3. Enqueue source vertex s.

4. Set visited[s] = TRUE.

5. Set parent[s] = -1.

6. While the queue is not empty do

a. Dequeue a vertex u.

b. For each vertex v
```

If visited[v] = FALSE AND rGraph[u][v] > 0 then

parent[v] = u.

visited[v] = TRUE.

Enqueue(v).

7. Return visited[t].

FORD_FULKERSON(graph[][], s, t)

1. Create the residual graph rGraph as a copy of graph.

2. Set max_flow = 0.

3. While BFS(rGraph, s, t, parent) returns TRUE do

a. Set path_flow = INFINITY.

b. Traverse from sink t to source s using parent[].

path_flow = MIN(path_flow, rGraph[parent[v]][v])

c. Traverse again from sink t to source s.

rGraph[parent[v]][v] =
rGraph[parent[v]][v] - path_flow

rGraph[v][parent[v]] =
rGraph[v][parent[v]] + path_flow

d. max_flow = max_flow + path_flow.

4. Return max_flow.

```
#include <stdio.h>
#include <limits.h>
#include <string.h>
#define V 6

int bfs(int rGraph[V][V], int s, int t, int parent[])
{
   int visited[V];
   memset(visited, 0, sizeof(visited));

   int queue[V], front = 0, rear = 0;
   queue[rear++] = s;
   visited[s] = 1;
   parent[s] = -1;

   while (front < rear)
   {
      int u = queue[front++];
      for (int v = 0; v < V; v++)
      {
         if (!visited[v] && rGraph[u][v] > 0)
         {
            queue[rear++] = v;
            parent[v] = u;
```

```
            visited[v] = 1;
          }
        }
      }
    return visited[t];
}

int fordFulkerson(int graph[V][V], int s, int t)
{
    int u, v;
    int rGraph[V][V];

    for (u = 0; u < V; u++)
        for (v = 0; v < V; v++)
            rGraph[u][v] = graph[u][v];

    int parent[V];
    int max_flow = 0;

    while (bfs(rGraph, s, t, parent))
    {
        int path_flow = INT_MAX;
        for (v = t; v != s; v = parent[v])
        {
            u = parent[v];
            if (rGraph[u][v] < path_flow)
                path_flow = rGraph[u][v];
        }

        for (v = t; v != s; v = parent[v])
        {
            u = parent[v];
            rGraph[u][v] -= path_flow;
            rGraph[v][u] += path_flow;
        }

        max_flow += path_flow;
    }
    return max_flow;
}

int main()
{
    int graph[V][V] = {
        {0, 16, 13, 0, 0, 0},
        {0, 0, 10, 12, 0, 0},
        {0, 4, 0, 0, 14, 0},
        {0, 0, 9, 0, 0, 20},
        {0, 0, 0, 7, 0, 4},
        {0, 0, 0, 0, 0, 0}
    };

    printf("Maximum flow = %d\n", fordFulkerson(graph, 0, 5));
    printf("\nTime Complexity: O(E x max_flow)\n");

    return 0;
}
```

Sample Input

Capacity matrix for a 6-vertex flow network, source = 0, sink = 5 (defined in code as graph[V][V])

Sample Output

Maximum flow = 23

Time Complexity

- Time Complexity: O(E x max_flow) for the basic Ford-Fulkerson method; $O(VE^2)$ when implemented with BFS (Edmonds-Karp variation)

13. Sum of Subsets Problem (Backtracking)

Aim

To implement the Sum of Subsets problem using the Backtracking method, to find subsets of a given set whose sum equals a given target value.

Algorithm

1. Start.
2. Read the set of n positive integers A[] and the target sum.
3. Define an array x[] of size n, where x[i] = 1 if the i-th element is included in the subset and x[i] = 0 otherwise.
4. Call SubsetSum(i = 0, currentSum = 0) recursively.
5. If currentSum equals target, print the elements of A[] for which x[i] = 1 as a valid solution, then return.
6. If i == n, or currentSum + remaining elements < target (cannot reach target), backtrack / return.
7. Otherwise, include A[i] in the subset: set x[i] = 1 and recursively call SubsetSum(i+1, currentSum + A[i]).
8. After returning, exclude A[i] from the subset: set x[i] = 0 and recursively call SubsetSum(i+1, currentSum) to explore the case without A[i].
9. This generates a state-space tree where each level corresponds to a decision (include / exclude) for one element.
10. Repeat until all the combinations have been explored.
11. Stop.

C Program

Pseudo Code

```
ALGORITHM: SUBSET SUM (Backtracking)

PRINT_SUBSET(x[], A[], n)

1. For j = 0 to n - 1

If x[j] = 1 then

Print A[j].

2. Return.


SUBSET_SUM(i, currentSum)

1. If currentSum = target then

Print the current subset.

Return.

2. If i = n OR currentSum > target then
```

Return.

3. Include A[i] in the subset.

x[i] = 1.

SUBSET_SUM(i + 1, currentSum + A[i]).

4. Exclude A[i] from the subset.

x[i] = 0.

SUBSET_SUM(i + 1, currentSum).

5. Return.

```
#include <stdio.h>

int A[] = {3, 5, 6, 7};
int target = 15;
int n = 4;
int x[4];

void printSubset()
{
   printf("Subset found: { ");
   for (int j = 0; j < n; j++)
      if (x[j])
         printf("%d ", A[j]);
   printf("}\n");
}

void subsetSum(int i, int currentSum)
{
   if (currentSum == target)
   {
      printSubset();
      return;
   }

   if (i == n || currentSum > target)
      return;

   /* include A[i] */
   x[i] = 1;
   subsetSum(i + 1, currentSum + A[i]);

   /* exclude A[i] */
   x[i] = 0;
   subsetSum(i + 1, currentSum);
}

int main()
{
   subsetSum(0, 0);
   printf("\nTime Complexity: O(2^n)\n");
   return 0;
}
```

Sample Input

Set A = {3, 5, 6, 7}, target = 15

Sample Output

Subset found: { 3 5 7 }

Time Complexity: O(2^n)

Time Complexity

- Time Complexity: O(2^n) (each element has two choices: include or exclude)

14. Hamiltonian Cycle (Backtracking)

Aim

To implement the Hamiltonian Cycle problem using the Backtracking method to determine whether a Hamiltonian cycle exists in a given graph, and to print the cycle if it exists.

Algorithm

1. Start.
2. Read the number of vertices V and the adjacency matrix graph[][] of the graph.
3. Initialize the path array path[] with -1 for all positions, and set path[0] = 0 (start the cycle from vertex 0).
4. Call HamCycleUtil(path, pos = 1) to recursively build the cycle.
5. If pos == V (all vertices have been included): check if there is an edge from the last vertex path[pos-1] to the starting vertex path[0]. If yes, the Hamiltonian cycle is complete; return true.
6. Otherwise, for each vertex v from 1 to V-1, check if it is safe to add v to the path using IsSafe(v, pos): v must be adjacent to the previously added vertex, and v must not already be present in the path.
7. If safe, add v to path[pos] and recursively call HamCycleUtil(path, pos+1).
8. If the recursive call succeeds, return true.
9. If it does not lead to a solution, remove v from the path (backtrack: path[pos] = -1) and try the next vertex.
10. If no vertex can be added at the current position, return false, causing backtracking to the previous position.
11. If a complete Hamiltonian cycle is found, print the cycle; otherwise report that no Hamiltonian cycle exists.
12. Stop.

C Program

Pseudo Code

```
ALGORITHM: HAMILTONIAN CYCLE (Backtracking)

IS_SAFE(v, pos)

1. If graph[path[pos - 1]][v] = 0 then

Return FALSE.

2. For i = 0 to pos - 1

If path[i] = v then

Return FALSE.

3. Return TRUE.
```

```
HAM_CYCLE_UTIL(pos)

1. If pos = V then

If graph[path[V - 1]][path[0]] = 1 then

Return TRUE.

Else

Return FALSE.

2. For v = 1 to V - 1

If IS_SAFE(v, pos) = TRUE then

path[pos] = v.

If HAM_CYCLE_UTIL(pos + 1) = TRUE then

Return TRUE.

path[pos] = -1. // Backtrack

3. Return FALSE.


HAM_CYCLE()

1. Initialize all elements of path[] to -1.

2. Set path[0] = 0.

3. If HAM_CYCLE_UTIL(1) = FALSE then

Print "No Hamiltonian Cycle Exists".

4. Else

Print the Hamiltonian Cycle by displaying
path[] followed by path[0].
```

```
#include <stdio.h>
#define MAX_V 20

int graph[MAX_V][MAX_V];
int path[MAX_V];
int V;

int isSafe(int v, int pos)
{
   if (graph[path[pos - 1]][v] == 0)
      return 0;
```

```
   for (int i = 0; i < pos; i++)
      if (path[i] == v)
         return 0;

   return 1;
}

int hamCycleUtil(int pos)
{
   if (pos == V)
      return graph[path[pos - 1]][path[0]] == 1;

   for (int v = 1; v < V; v++)
   {
      if (isSafe(v, pos))
      {
         path[pos] = v;
         if (hamCycleUtil(pos + 1))
            return 1;
         path[pos] = -1;
      }
   }
   return 0;
}

int hamCycle()
{
   for (int i = 0; i < V; i++)
      path[i] = -1;
   path[0] = 0;

   if (!hamCycleUtil(1))
   {
      printf("\nNo Hamiltonian cycle exists\n");
      return 0;
   }

   printf("\nHamiltonian cycle: \n");
   for (int i = 0; i < V; i++)
      printf("%d ", path[i]);
   printf("%d\n", path[0]);
   return 1;
}

int main()
{
   printf("Enter number of vertices: ");
   scanf("%d", &V);

   printf("Enter adjacency matrix: \n");
   for (int i = 0; i < V; i++)
      for (int j = 0; j < V; j++)
         scanf("%d", &graph[i][j]);

   hamCycle();
```

```
    printf("\nTime Complexity: O(N!)\n");

    return 0;
}
```

Sample Input

Enter number of vertices: 5
Enter adjacency matrix:
0 1 0 1 0
1 0 1 1 1
0 1 0 0 1
1 1 0 0 1
0 1 1 1 0

Sample Output

Hamiltonian cycle:
0 1 2 4 3 0

Time Complexity: O(N!)

Time Complexity

- Time Complexity: O(N!) (worst case, with backtracking pruning unsafe branches early)

15. Job Sequencing with Deadlines (Greedy Method)

Aim

To implement the Job Sequencing with Deadlines problem using the Greedy method, to select and schedule jobs so that the total profit is maximized, given that each job takes one unit of time and has a deadline.

Algorithm

1. Start.
2. Read the number of jobs n, and for each job its deadline and profit.
3. Sort all the jobs in descending order of their profit.
4. Find the maximum deadline among all the jobs; this determines the number of time slots available.
5. Create a slot/result array of size equal to the maximum deadline, and initialize all slots as free (empty).
6. For each job (taken in the sorted, descending-profit order): try to place the job in the latest available free slot that is less than or equal to its own deadline (search from its deadline down to slot 1).
7. If a free slot is found, assign the job to that slot, mark the slot as occupied, and add the job's profit to the total profit.
8. If no free slot is available within the job's deadline, skip the job.
9. Repeat for all jobs.
10. Display the sequence of selected jobs and the total profit earned.
11. Stop.

C Program

Pseudo Code

```
ALGORITHM: JOB SEQUENCING WITH DEADLINES (Greedy Method)

JOB_SEQUENCING(job[], deadline[], profit[], n)

1. Sort all jobs in descending order of profit.

2. Find the maximum deadline among all jobs.

3. Create an array slot[1...maxDeadline] and initialize
all slots to EMPTY.

4. Set totalProfit = 0.

5. For each job in the sorted list

For j = MIN(deadline[i], maxDeadline) down to 1

If slot[j] is EMPTY then
```

```
slot[j] = job[i].

totalProfit = totalProfit + profit[i].

Break.

6. Print the selected jobs in their assigned slots.

7. Print the total profit.
```

```
#include <stdio.h>

struct Job
{
   char id;
   int deadline;
   int profit;
};

void sortJobs(struct Job jobs[], int n)
{
   for (int i = 0; i < n - 1; i++)
      for (int j = i + 1; j < n; j++)
         if (jobs[j].profit > jobs[i].profit)
         {
            struct Job temp = jobs[i];
            jobs[i] = jobs[j];
            jobs[j] = temp;
         }
}

void jobSequencing(struct Job jobs[], int n)
{
   sortJobs(jobs, n);

   int maxDeadline = jobs[0].deadline;
   for (int i = 1; i < n; i++)
      if (jobs[i].deadline > maxDeadline)
         maxDeadline = jobs[i].deadline;

   int slot[maxDeadline + 1];
   char result[maxDeadline + 1];
   for (int i = 1; i <= maxDeadline; i++)
      slot[i] = 0;

   int totalProfit = 0;

   for (int i = 0; i < n; i++)
   {
      for (int j = jobs[i].deadline; j > 0; j--)
      {
         if (slot[j] == 0)
         {
            slot[j] = 1;
            result[j] = jobs[i].id;
            totalProfit += jobs[i].profit;
            break;
```

```
            }
        }
    }

    printf("Selected Jobs: ");
    for (int i = 1; i <= maxDeadline; i++)
        if (slot[i])
            printf("%c ", result[i]);

    printf("\nTotal Profit: %d\n", totalProfit);
}

int main()
{
    struct Job jobs[] = {
        {'A', 2, 100},
        {'B', 1, 19},
        {'C', 2, 27},
        {'D', 1, 25},
        {'E', 3, 15}
    };
    int n = 5;

    jobSequencing(jobs, n);

    printf("\nTime Complexity: O(n^2)\n");

    return 0;
}
```

Sample Input

Jobs: A(deadline=2, profit=100), B(deadline=1, profit=19), C(deadline=2, profit=27), D(deadline=1, profit=25), E(deadline=3, profit=15)

Sample Output

Selected Jobs: C A E
Total Profit: 142

Time Complexity

- Time Complexity: $O(n^2)$ (O(n log n) for sorting jobs by profit, plus O(n x d) for slot allocation)

16. Topological Sorting (using DFS / Stack)

Aim

To implement Topological Sorting of a given Directed Acyclic Graph (DAG) using the Depth First Search (DFS) based method.

Algorithm

1. Start.
2. Read the number of vertices and the adjacency list / matrix of the directed graph.
3. Initialize a visited[] array with all values set to false (0), and an empty stack to hold the topological order.
4. For each vertex v in the graph that is not yet visited, call TopologicalSortUtil(v).
5. In TopologicalSortUtil(v): mark v as visited.
6. Recursively visit all the adjacent (neighbouring) vertices of v that have not yet been visited, by calling TopologicalSortUtil on each of them.
7. After all the neighbours of v have been recursively processed, push v onto the stack (this ensures v is placed after all of its dependent / successor vertices).
8. Repeat the DFS process for every unvisited vertex in the graph until all vertices have been visited.
9. Once all vertices are processed, pop all elements from the stack; the order in which they are popped gives the topological order of the graph.
10. Display the topological order.
11. Stop.

C Program

Pseudo Code

```
ALGORITHM: TOPOLOGICAL SORTING (Using DFS / Stack)

TOPO_SORT_UTIL(v)

1. Set visited[v] = TRUE.

2. For each adjacent vertex u of v

If visited[u] = FALSE then

TOPO_SORT_UTIL(u).

3. Push vertex v onto the stack.

4. Return.


TOPOLOGICAL_SORT()
```

```
1. Initialize all elements of visited[] to FALSE.

2. Create an empty stack.

3. For each vertex v = 0 to n - 1

If visited[v] = FALSE then

TOPO_SORT_UTIL(v).

4. Print "Topological Order:".

5. While the stack is not empty

Print POP(stack).
```

```
#include <stdio.h>
#define MAX 20

int adj[MAX][MAX];
int visited[MAX];
int stack[MAX], top = -1;
int n;

void push(int v)
{
   stack[++top] = v;
}

void topologicalSortUtil(int v)
{
   visited[v] = 1;

   for (int i = 0; i < n; i++)
      if (adj[v][i] && !visited[i])
         topologicalSortUtil(i);

   push(v);
}

void topologicalSort()
{
   for (int i = 0; i < n; i++)
      visited[i] = 0;

   for (int v = 0; v < n; v++)
      if (!visited[v])
         topologicalSortUtil(v);

   printf("\nTopological Sorting Order: \n");
   while (top != -1)
      printf("%d ", stack[top--]);
   printf("\n");
}

int main()
{
```

```
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix: \n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    topologicalSort();

    printf("\nTime Complexity: O(V + E)\n");

    return 0;
}
```

Sample Input

Enter number of vertices: 6
Enter adjacency matrix:
(matrix for graph with vertices a,b,c,d,e,f and edges a-c, a-f, b-c, b-g, d-f, d-g)

Sample Output

Topological Sorting Order:
d a c b g f

Time Complexity: O(V + E)

Time Complexity

- Time Complexity: O(V + E) where V = number of vertices, E = number of edges

17. Graph Traversal using BFS and DFS

Aim

To traverse a given graph using Breadth First Search (BFS) and Depth First Search (DFS) methods and print the order in which vertices are visited.

Algorithm

1. Start.
2. Read the number of vertices and the adjacency matrix graph[][] of the graph, and the starting vertex s.
3. BFS(s): initialize a visited[] array to false for all vertices, and an empty queue.
4. Mark s as visited and enqueue s.
5. While the queue is not empty: dequeue a vertex u and print/visit it.
6. For every vertex v adjacent to u, if v is not visited, mark it visited and enqueue it.
7. Repeat until the queue becomes empty; this gives the BFS traversal order.
8. DFS(s): initialize a visited[] array to false for all vertices.
9. Mark the current vertex s as visited and print/visit it.
10. For every vertex v adjacent to s, if v is not visited, recursively call DFS(v).
11. Repeat recursively until all reachable vertices from s have been visited; this gives the DFS traversal order.
12. Stop.

C Program

Pseudo Code

```
ALGORITHM: TRAVELLING SALESMAN PROBLEM (Dynamic Programming)

TSP(cost[][], n)

1. Set VISITED_ALL = (1 << n) - 1.

2. Initialize all dp[mask][i] = INFINITY.

3. Set dp[1][0] = 0. // Start from city 0.

4. For mask = 1 to VISITED_ALL

For each city i where bit i is set in mask

If dp[mask][i] = INFINITY then
```

Continue.

For each city j not included in mask

newMask = mask OR (1 << j).

newCost = dp[mask][i] + cost[i][j].

If newCost < dp[newMask][j] then

dp[newMask][j] = newCost.

5. Set minCost = INFINITY.

6. For each city i = 1 to n - 1

If dp[VISITED_ALL][i] + cost[i][0] < minCost then

minCost = dp[VISITED_ALL][i] + cost[i][0].

7. Return minCost.

```
#include <stdio.h>
#define MAX 20

int adj[MAX][MAX];
int visitedBFS[MAX], visitedDFS[MAX];
int n;

void bfs(int s)
{
   int queue[MAX], front = 0, rear = 0;
   for (int i = 0; i < n; i++)
      visitedBFS[i] = 0;

   queue[rear++] = s;
   visitedBFS[s] = 1;

   printf("BFS Traversal: ");
   while (front < rear)
   {
      int u = queue[front++];
      printf("%d ", u);

      for (int v = 0; v < n; v++)
      {
         if (adj[u][v] && !visitedBFS[v])
         {
            visitedBFS[v] = 1;
            queue[rear++] = v;
```

```
        }
      }
    }
    printf("\n");
}

void dfsUtil(int u)
{
    visitedDFS[u] = 1;
    printf("%d ", u);

    for (int v = 0; v < n; v++)
        if (adj[u][v] && !visitedDFS[v])
            dfsUtil(v);
}

void dfs(int s)
{
    for (int i = 0; i < n; i++)
        visitedDFS[i] = 0;

    printf("DFS Traversal: ");
    dfsUtil(s);
    printf("\n");
}

int main()
{
    int start;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix: \n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    bfs(start);
    dfs(start);

    printf("\nTime Complexity: O(V + E)\n");

    return 0;
}
```

Sample Input

Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0

1 0 0 1 1
0 1 1 0 1
0 0 1 1 0
Enter starting vertex: 0

Sample Output

BFS Traversal: 0 1 2 3 4
DFS Traversal: 0 1 3 2 4

Time Complexity

- Time Complexity: O(V + E) for both BFS and DFS, where V = number of vertices, E = number of edges
- Space Complexity: O(V)

18. Topological Sorting using BFS (Kahn's Algorithm)

Aim

To implement Topological Sorting of a given Directed Acyclic Graph (DAG) using the Breadth First Search based method (Kahn's Algorithm).

Algorithm

1. Start.
2. Read the number of vertices and the adjacency matrix/list of the directed graph.
3. Compute the in-degree of every vertex (the number of incoming edges) by scanning all the edges of the graph.
4. Initialize an empty queue and enqueue all the vertices whose in-degree is 0 (vertices with no dependencies).
5. Initialize an empty list/array to store the topological order, and a counter visitedCount = 0.
6. While the queue is not empty: dequeue a vertex u, append u to the topological order list, and increment visitedCount.
7. For every vertex v adjacent to u (i.e., every edge u -> v), decrement the in-degree of v by 1.
8. If the in-degree of v becomes 0 after decrementing, enqueue v.
9. Repeat until the queue becomes empty.
10. If visitedCount equals the total number of vertices, the topological order list gives a valid topological sort; otherwise, the graph contains a cycle and no topological ordering exists.
11. Display the topological order.
12. Stop.

C Program

Pseudo Code

```
ALGORITHM: TOPOLOGICAL SORTING USING BFS (KAHN'S ALGORITHM)

TOPOLOGICAL_SORT_BFS(adj[][], n)

1. Initialize indegree[0...n-1] = 0.

2. For each vertex i = 0 to n - 1

For each vertex j = 0 to n - 1

If adj[i][j] = 1 then

indegree[j] = indegree[j] + 1.
```

3. Create an empty queue.

4. For each vertex i = 0 to n - 1

If indegree[i] = 0 then

Enqueue(i).

5. While the queue is not empty

a. Dequeue a vertex u.

b. Print u.

c. For each adjacent vertex v of u

If adj[u][v] = 1 then

indegree[v] = indegree[v] - 1.

If indegree[v] = 0 then

Enqueue(v).

6. End.

```
#include <stdio.h>
#define MAX 20

int adj[MAX][MAX];
int n;

void topologicalSortBFS()
{
   int inDegree[MAX] = {0};
   int queue[MAX], front = 0, rear = 0;
   int topOrder[MAX], count = 0;

   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         if (adj[i][j])
            inDegree[j]++;

   for (int i = 0; i < n; i++)
      if (inDegree[i] == 0)
         queue[rear++] = i;

   while (front < rear)
```

```
    {
        int u = queue[front++];
        topOrder[count++] = u;

        for (int v = 0; v < n; v++)
        {
            if (adj[u][v])
            {
                inDegree[v]--;
                if (inDegree[v] == 0)
                    queue[rear++] = v;
            }
        }
    }

    if (count != n)
    {
        printf("\nGraph has a cycle, topological sort not possible\n");
        return;
    }

    printf("\nTopological Sorting Order (Kahn's Algorithm): \n");
    for (int i = 0; i < count; i++)
        printf("%d ", topOrder[i]);
    printf("\n");
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix: \n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    topologicalSortBFS();

    printf("\nTime Complexity: O(V + E)\n");

    return 0;
}
```

Sample Input

Enter number of vertices: 6
Enter adjacency matrix:
(matrix for graph with vertices a,b,c,d,e,f and edges a-c, a-f, b-c, b-g, d-f, d-g)

Sample Output

Topological Sorting Order (Kahn's Algorithm):
d a b c f g

Time Complexity

- Time Complexity: O(V + E) where V = number of vertices, E = number of edges
- Space Complexity: O(V)

19. Travelling Salesman Problem (Dynamic Programming)

Aim

To find the shortest possible route that visits every city exactly once and returns to the starting city, using Dynamic Programming (Held-Karp algorithm) to solve the Travelling Salesman Problem.

Algorithm

1. Start.
2. Read the number of cities n and the cost adjacency matrix cost[][], where cost[i][j] is the distance/cost from city i to city j.
3. Define dp[mask][i] as the minimum cost to visit all the cities in the set 'mask' (a bitmask representing the subset of visited cities), ending at city i.
4. Initialize dp[1][0] = 0 (start at city 0 with only city 0 visited, represented by mask = 1), and all other dp values as infinity.
5. For every subset 'mask' of cities (from 1 to 2^n - 1) that includes the starting city, and for every city i in that subset: if dp[mask][i] is not infinity, try extending the path to every city j not yet in the subset.
6. Update dp[mask | (1<<j)][j] = min(dp[mask | (1<<j)][j], dp[mask][i] + cost[i][j]).
7. Repeat for all subsets and all valid (i, j) pairs, building up the DP table.
8. After processing all subsets, compute the final answer as the minimum over all cities i (i != 0) of dp[(1<<n) - 1][i] + cost[i][0], which accounts for returning to the starting city.
9. Display the minimum cost of the optimal tour.
10. Stop.

C Program

```
#include <stdio.h>
#include <limits.h>
#define MAX 15
#define INF INT_MAX / 2

int n;
int cost[MAX][MAX];
int dp[1 << MAX][MAX];

int tsp()
{
   int VISITED_ALL = (1 << n) - 1;

   for (int mask = 0; mask <= VISITED_ALL; mask++)
      for (int i = 0; i < n; i++)
         dp[mask][i] = INF;

   dp[1][0] = 0;

   for (int mask = 1; mask <= VISITED_ALL; mask++)
   {
      for (int i = 0; i < n; i++)
      {
```

```
            if (!(mask & (1 << i)) || dp[mask][i] == INF)
                continue;

            for (int j = 0; j < n; j++)
            {
                if (mask & (1 << j))
                    continue;

                int newMask = mask | (1 << j);
                int newCost = dp[mask][i] + cost[i][j];
                if (newCost < dp[newMask][j])
                    dp[newMask][j] = newCost;
            }
        }
    }

    int minCost = INF;
    for (int i = 1; i < n; i++)
        if (dp[VISITED_ALL][i] != INF)
            if (dp[VISITED_ALL][i] + cost[i][0] < minCost)
                minCost = dp[VISITED_ALL][i] + cost[i][0];

    return minCost;
}

int main()
{
    printf("Enter number of cities: ");
    scanf("%d", &n);

    printf("Enter the cost adjacency matrix: \n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    int result = tsp();
    printf("\nMinimum cost of the Travelling Salesman tour = %d\n", result);

    printf("\nTime Complexity: O(n^2 * 2^n)\n");
    printf("Space Complexity: O(n * 2^n)\n");

    return 0;
}
```

Sample Input

Enter number of cities: 4
Enter the cost adjacency matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

Sample Output

Minimum cost of the Travelling Salesman tour = 80

Time Complexity

- Time Complexity: $O(n^2 \times 2^n)$ using the Held-Karp dynamic programming approach
- Space Complexity: $O(n \times 2^n)$
- (Brute-force approach without DP: O(n!))

ALGORITHM: TRAVELLING SALESMAN PROBLEM (Dynamic Programming)

TSP()

1. Set VISITED_ALL = (1 << n) - 1.

2. Initialize all elements of dp[mask][i] to INFINITY.

3. Set dp[1][0] = 0. // Start from city 0.

4. For each mask from 1 to VISITED_ALL

For each city i

If city i is not included in mask OR
dp[mask][i] = INFINITY

Continue.

For each city j

If city j is already included in mask

Continue.

newMask = mask OR (1 << j).

newCost = dp[mask][i] + cost[i][j].

If newCost < dp[newMask][j] then

dp[newMask][j] = newCost.

5. Set minCost = INFINITY.

6. For each city i = 1 to n - 1

If dp[VISITED_ALL][i] + cost[i][0] < minCost then

minCost = dp[VISITED_ALL][i] + cost[i][0].

7. Return minCost.